

Sistemas Multiagente

Juegos Alternados Minimax - Poda - Funciones de evaluación

Sergio Yovine

Universidad ORT Uruguay
yovine@ort.edu.uy

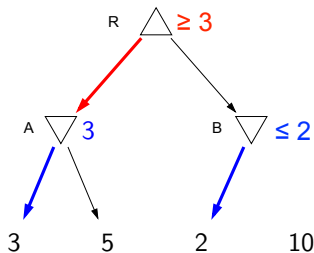
6 de mayo de 2026

Minimax

- ▶ Requiere recorrer el árbol de juego
- ▶ Complejidad en tiempo: $O(b^d)$
 - ▶ *factor de ramificación* b
 - ▶ *cantidad de movimientos (o profundidad)* d
- ▶ Los árboles pueden ser muy grandes
 - ▶ En Ajedrez $b \sim 35$ y $d \sim 50$
 - ▶ En Go b puede llegar hasta 361 y $d \sim 100$
 - ▶ ¿En Ta-Te-Ti?
- ▶ En general es difícil evitar recorrer el árbol completo
- ▶ Técnicas para acelerar la ejecución
 - ▶ Poda
 - ▶ Estimar el valor mediante *funciones de evaluación*

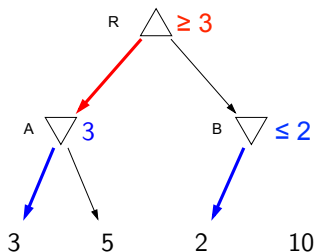
Poda $\alpha - \beta$

Algoritmo de tipo *branch-and-bound*



- ▶ El valor de la raíz $R = \max\{A, B\} = \max\{\min\{3, 5\}, \min\{2, 10\}\}$
- ▶ Después de evaluar A sabemos que $R \geq 3$
- ▶ Después de evaluar el hijo izquierdo de B sabemos que $B \leq 2$
- ▶ $R = \max\{\geq 3, \leq 2\} = \geq 3$
- ▶ Se puede **podar** el resto del subárbol B

Poda $\alpha - \beta$



- a_s es una cota inferior del valor del nodo MAX s

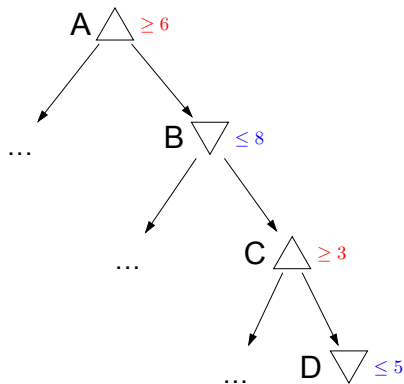
$$a_R = 3$$

- b_s es una cota superior del valor del nodo MIN s

$$b_A = 3$$

$$b_B = 2$$

Poda $\alpha - \beta$



El *camino óptimo* es el camino de la política *minimax*

En el camino óptimo el valor en todos los nodos es el mismo e igual al valor de *minimax*

$$\alpha_s = \max_{s' \in \text{ancestros}(s) \cup \{s\}} a_{s'}$$

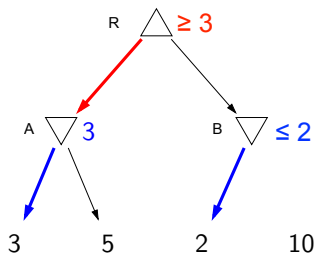
$$\beta_s = \min_{s' \in \text{ancestros}(s) \cup \{s\}} b_{s'}$$

- ▶ Si $\geq \alpha_s$ y $\leq \beta_s$ no tienen una intersección no trivial entonces podar s
- ▶ Ejemplo: $\alpha_D = \max\{6, 3\} = 6$ y $\beta_D = \min\{8, 5\} = 5$

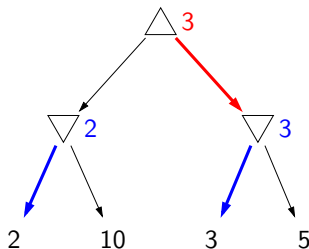
El algoritmo de poda α - β calcula correctamente $V_{\pi_{\text{minimax}}, \pi_{\text{min}}}$

Poda $\alpha - \beta$

La mejora computacional depende del orden de los árboles



Se poda B



No se poda B

- ▶ Peor caso $O(b^{2d})$
- ▶ Mejor orden $O(b^d)$
- ▶ Orden aleatorio $O(b^{\frac{3}{2}d})$

Funciones de evaluación

Limitar la profundidad del árbol a explorar.

Calcular $V_{\pi_{\text{minimax}}, \pi_{\text{min}}}(s, d_{\text{max}})$, $d_{\text{max}} > 0$, con:

$$V_{\pi_{\text{minimax}}, \pi_{\text{min}}}(s, d) =$$

$$\begin{cases} \text{Utilidad}(s) & \text{EsFinal}(s) \\ \text{Eval}(s) & d = 0 \\ \max_{a \in \text{Acciones}(s)} V_{\pi_{\text{minimax}}, \pi_{\text{min}}}(Suc(s, a), d - 1) & \text{Jugador}(s) = \text{Agente} \\ \min_{a \in \text{Acciones}(s)} V_{\pi_{\text{minimax}}, \pi_{\text{min}}}(Suc(s, a), d - 1) & \text{Jugador}(s) = \text{Oponente} \end{cases}$$

$\text{Eval}(s)$ es una estimación de $V_{\pi_{\text{minimax}}, \pi_{\text{min}}}(s)$.

d_{max} es la máxima cantidad de movimientos.

Este algoritmo calcula de manera aproximada $V_{\pi_{\text{minimax}}, \pi_{\text{min}}}$

Funciones de evaluación

Limitar la profundidad del árbol a explorar.

Calcular $V_{\pi_{\text{max}}, \pi_{\text{min}}}(s, d_{\text{max}})$, $d_{\text{max}} > 0$, con:

$$V_{\pi_{\text{max}}, \pi_{\text{min}}}(s, d) =$$

$$\begin{cases} \text{Utilidad}(s) & \text{EsFinal}(s) \\ \text{Eval}(s) & d = 0 \\ \max_{a \in \text{Acciones}(s)} V_{\pi_{\text{max}}, \pi_{\text{min}}}(Suc(s, a), d) & \text{Jugador}(s) = \text{Agente} \\ \min_{a \in \text{Acciones}(s)} V_{\pi_{\text{max}}, \pi_{\text{min}}}(Suc(s, a), d - 1) & \text{Jugador}(s) = \text{Oponente} \end{cases}$$

$\text{Eval}(s)$ es una estimación de $V_{\pi_{\text{max}}, \pi_{\text{min}}}(s)$.

d_{max} es la máxima cantidad de rondas.

Este algoritmo calcula de manera aproximada $V_{\pi_{\text{max}}, \pi_{\text{min}}}$

Funciones de evaluación

¿Cómo diseñamos una *buena* función de evaluación?

1. *Eval* debe ordenar los estados terminales de la misma manera que la función de utilidad verdadera

$$Eval(win) > Eval(draw) > Eval(loss)$$

De lo contrario, el agente podría equivocarse incluso si puede ver todo el camino hasta el final.

2. El cálculo de *Eval* no debe tardar demasiado, dado que el objetivo es buscar más rápido.
3. Para todo estado no terminal s , $Eval(s)$ debe estar *fuertemente correlacionada* con las *posibilidades reales de ganar* a partir de s .

Funciones de evaluación

En **Ajedrez**, a las piezas se les otorga un *valor numérico* (relativo o material), representativo de su importancia

- ▶ Peón: 1
- ▶ Caballo y Alfil: 3
- ▶ Torre: 5
- ▶ Dama: 9

A partir de esto, podríamos definir la función de evaluación siguiente:

$$\begin{aligned} Eval(s) = & 9 \left(D_a(s) - D_o(s) \right) + 5 \left(T_a(s) - T_o(s) \right) \\ & + 3 \left(C_a(s) - C_o(s) + A_a(s) - A_o(s) \right) \\ & + 1 \left(P_a(s) - P_o(s) \right) \end{aligned}$$

$D(s), \dots$ es la cantidad de damas, etc., de **agente** y **oponente**

Funciones de evaluación

La función anterior no ordena los estados terminales correctamente. Si $D_p(s) = D_p(s')$, etc. para $p \in \{a, o\}$, se tiene $Eval(s) = Eval(s')$, pero:

- ▶ $R_a(s) = 1$ y $R_o(s) = 0$ agente gana
- ▶ $R_a(s) = 0$ y $R_o(s) = 1$ agente pierde

Redefinimos $Eval$ de la siguiente manera:

$$\begin{aligned} Eval(s) = & \aleph \left(R_a(s) - R_o(s) \right) \\ & + 9 \left(D_a(s) - D_o(s) \right) + 5 \left(T_a(s) - T_o(s) \right) \\ & + 3 \left(C_a(s) - C_o(s) + A_a(s) - A_o(s) \right) \\ & + 1 \left(P_a(s) - P_o(s) \right) \end{aligned}$$

$\aleph \in \mathbb{R}^+$ es un número grande

Funciones de evaluación

Enfoque general: definir *Eval* como una *función lineal paramétrica*

$$Eval(s; \mathbf{w}) = \mathbf{w} \cdot \phi(s) = \sum_{k=1}^K w_k \phi_k(s) \quad \mathbf{w} \in \mathbb{R}^K$$

$\phi(s)$ es un vector de *features*.

Ejemplo: Ajedrez

$$\phi_1(s) = R_{\textcolor{red}{a}}(s) - R_{\textcolor{blue}{o}}(s)$$

$$\phi_2(s) = D_{\textcolor{red}{a}}(s) - D_{\textcolor{blue}{o}}(s)$$

$$\phi_3(s) = T_{\textcolor{red}{a}}(s) - T_{\textcolor{blue}{o}}(s)$$

$$\phi_4(s) = C_{\textcolor{red}{a}}(s) - C_{\textcolor{blue}{o}}(s) + A_{\textcolor{red}{a}}(s) - A_{\textcolor{blue}{o}}(s)$$

$$\phi_5(s) = P_{\textcolor{red}{a}}(s) - P_{\textcolor{blue}{o}}(s)$$

$$Eval(s) = (\aleph, 9, 5, 3, 1) \cdot \phi(s)$$

Bibliografía

- ▶ S. Russell, P. Norvig. *Artificial Intelligence. A Modern Approach*. 4th ed. Prentice Hall. Capítulo 5.